

Lostfound+

INTRODUCTION:

LostFound+ is a versatile and user-friendly software platform designed to streamline the recovery and management of campus lost and found property. Suitable for wallets, ID cards, electronics, keys, and more, it offers structured reporting fields—including category, color, campus location, and description—that can be quickly submitted using a clean and intuitive web interface. Users can easily discover misplaced belongings using dynamic multi-criteria filters, while an evidence-based 100-point smart match engine automatically pairs opposing reports with high precision. To prevent fraud, blind proof claims and role-based officer logins ensure property handovers are securely authenticated by campus authorities. With built-in data visualization tools, administrators can track live recovery rates, category trends, and campus incident hotspots in real time. Its transparency, ease of use, and automated matching make it an ideal choice for colleges, universities, and security teams seeking an efficient property recovery solution.

SCENARIO Based Case Study: LostFound+

Meet Alex, a busy university student with a demanding academic schedule who values organization and peace of mind on a bustling campus. Alex regularly moves between lectures, study halls, and the campus library, carrying everyday essentials like his student ID card, wallet, keys, and transit passes. While studying on the second floor of the Central Library, Alex accidentally misplaces his black leather wallet—putting him at risk of losing crucial identification and campus access.

Instead of turning to chaotic WhatsApp group chats or unmonitored bulletin boards, Alex discovers **LostFound+**, an intelligent campus property recovery platform designed to streamline lost and found operations. Intrigued by its centralized system and smart recovery tools, Alex turns to LostFound+ to recover his misplaced belongings.

- **Streamlined Incident Reporting:** Alex navigates to LostFound+ and clicks "Report Item." He selects the "I Lost An Item" form, entering specific descriptive details: category (*Wallet*), color (*Black*), precise campus venue (*Central Library 2nd Floor*), date, and key notes about the contents. In seconds, his report is published to the cloud with an active "Open" status.
- **Campus Discovery Feed & Live Filters:** Meanwhile, students and campus staff can access the central "Browse" catalog. Alex can quickly filter active items across campus

by category pills, venue keywords, and report types (*Lost* vs. *Found*). Clean status indicators and fast, responsive filters ensure items never get lost in disorganized message threads.

- **100-Point Deterministic Smart Match Engine:** A few hours later, a fellow student finds the wallet in the library study area and logs it as "Found." Rather than storing records passively, LostFound+'s deterministic 100-point algorithm immediately evaluates both listings against physical criteria—allocating weighted points for category alignment (30 pts), location proximity (30 pts), color match (20 pts), keyword overlap (10 pts), and date proximity (10 pts). The system instantly pairs the reports with an 85% match confidence and displays transparent evidence chips explaining the score.
- **Blind Proof Claim Submission:** Seeing the high-scoring match, Alex initiates an ownership claim directly on the found item's card. To protect his privacy and prevent theft, he submits blind verification details only the rightful owner would know—citing his exact student registration number (2024-8891) and a private transit pass tucked inside. Submitting the claim immediately locks the listing into a "Claimed" status, preventing unauthorized claims.
- **Role-Based Handover & Security Desk Verification:** To eliminate fraud, student users cannot approve their own claims. The claim is routed to the campus security office as "Awaiting Security Desk Review." When Alex visits the desk, an authorized officer logs into **Security Desk Authority Mode**, cross-examines Alex's physical credentials against his submitted proof, and executes an authorized "Approve (Hand Over)." The item is safely marked as "Returned" in the database.
- **Real-Time Analytics & Hotspot Dashboard:** Behind the scenes, campus administrators and security personnel leverage the Operations Dashboard powered by native MongoDB aggregations. They monitor live KPIs, item category distributions, recovery resolution rates, and high-incident loss hotspots (such as the Central Library and Cafeteria) to optimize campus patrol routines and safety protocols.

Alex's Experience: Thanks to LostFound+, Alex recovered his lost wallet and vital documents within hours, entirely avoiding the stress of lost identity cards and administrative replacement fees. The platform eliminated manual guesswork through automated matching, protected his property with verified security handover, and kept the entire campus connected through a single transparent portal. LostFound+ proved to be an indispensable campus utility, restoring Alex's peace of mind with speed, precision, and confidence.

SYSTEM REQUIREMENTS: LostFound+

To ensure smooth development, deployment, and usage of the **LostFound+** Web Application, certain system prerequisites must be met. These requirements are categorized into software,

setup, and hardware specifications, all of which contribute to building a robust, responsive, and secure campus property recovery platform.

1. Software Requirements

These are the essential tools, libraries, and runtime platforms required to develop, test, and run the LostFound+ application efficiently.

- **Operating System:** Windows 10/11, macOS, or Linux — Provides a stable cross-platform environment for local development and build pipelines.
 - **Node.js (v18 or higher):** Powers the server-side JavaScript runtime environment, handles RESTful API endpoints, and orchestrates frontend build tooling via Vite.
 - **npm (v9 or higher):** Node Package Manager used for installing and maintaining dependencies across both the React frontend and Express backend.
 - **React.js & Vite:** Modern frontend library bundled with Vite to provide hot-module replacement, high-performance compilation, and responsive UI rendering.
 - **Tailwind CSS:** Utility-first CSS framework for crafting a modern, clean, and accessible user interface with responsive mobile/desktop layouts.
 - **Express.js:** Lightweight and flexible backend web application framework for Node.js, managing API routing, 100-point matching algorithms, and claim status transitions.
 - **MongoDB Atlas & Mongoose ODM:** Cloud-hosted NoSQL document database utilizing Mongoose for schema modeling, transactional updates, and native aggregation pipelines (KPI metrics, category charts, and incident hotspots).
 - **Modern Web Browser:** Google Chrome, Mozilla Firefox, Microsoft Edge, or Safari (latest stable versions) — Required for testing UI responsiveness, local storage, and security session modals.
 - **Version Control (Git & GitHub):** Version management tool used for codebase collaboration, branch maintenance, and source tracking.
 - **Code Editor:** Visual Studio Code (VS Code) — Preferred Integrated Development Environment (IDE) configured with ESLint, Prettier, and terminal support.
 - **API Testing Tool (Optional):** Postman or Thunder Client — Used to debug and validate HTTP request/response payloads across the `/api/items`, `/api/matches`, `/api/claims`, and `/api/dashboard` endpoints.
-

2. Hardware Requirements

Describes the minimum and recommended hardware specifications needed to support smooth development, compilation, and concurrent execution of frontend and backend servers.

- **Processor (CPU):**
 - *Minimum:* Intel Core i3 (7th Gen or above) / AMD Ryzen 3
 - *Recommended:* Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or Apple Silicon (M-Series) — Ensures smooth simultaneous compilation of Vite and Node runtime servers.
- **RAM (Memory):**
 - *Minimum:* 4 GB free RAM
 - *Recommended:* 8 GB to 16 GB — Needed for smoothly running the IDE, local development servers (`npm run dev` and `node server.js`), background database connections, and multiple browser tabs.
- **Storage Space:**
 - *Minimum:* 1.5 GB free disk space — Required for `node_modules` installations across client and server packages, build caches, and project asset storage.
 - *Recommended:* SSD (Solid-State Drive) for faster build speeds and quick server startup.
- **Display Resolution:**
 - *Minimum:* 1366 × 768 pixels
 - *Recommended:* 1920 × 1080 (Full HD) or higher — Recommended for optimal inspection of multi-column detail views, dashboard charts (Recharts), and split-terminal workflows.
- **Network Connectivity:**
 - Stable broadband Internet connection — Required for active connections to the cloud-hosted MongoDB Atlas cluster and package downloads.

PROJECT ARCHITECTURE: LostFound+

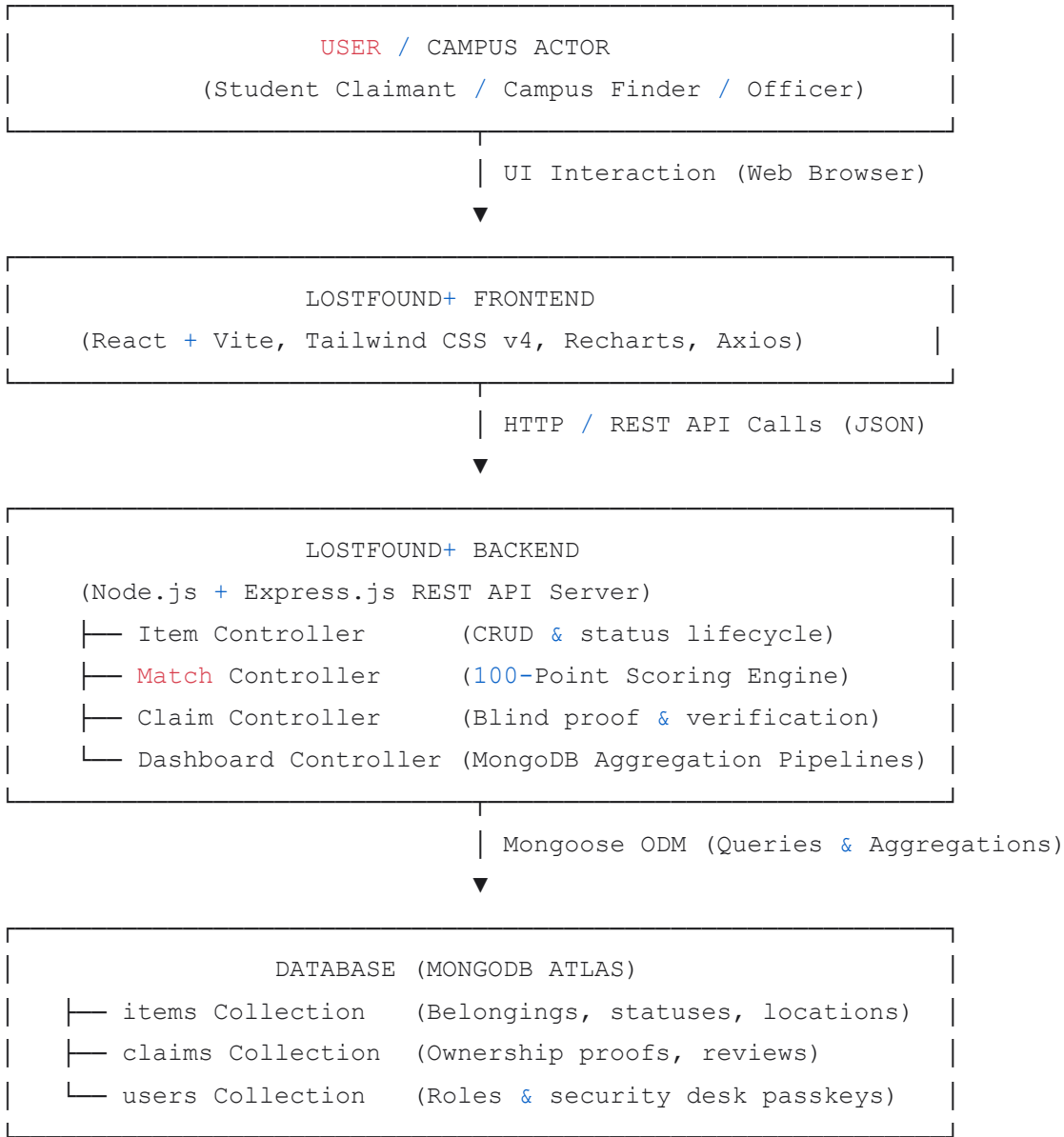
TECHNICAL ARCHITECTURE:

In this architecture, the workflow begins with a campus user (represented by the **"User / Student"** or **"Security Officer"** node) interacting with the LostFound+ web interface (represented by the **"LostFound+ Frontend"** node). The frontend is built using React, Vite, and Tailwind CSS, providing an interactive, accessible user interface for reporting items, filtering feeds, submitting blind claims, and viewing analytics charts.

When an action is initiated—such as reporting a lost wallet, querying discovery filters, or calculating smart matches—the LostFound+ Frontend issues asynchronous REST API requests

over HTTP/JSON using an Axios client to the LostFound+ backend (represented by the "LostFound+ Backend" node).

codeCode



The **LostFound+ Backend**, built with Node.js and Express.js, receives and routes these requests through specialized controller modules:

- Item Controller:** Handles CRUD operations, category validation, and status lifecycle state transitions (Open → Claimed → Returned).
- Match Controller:** Executes the **100-Point Deterministic Smart Match Engine**, evaluating opposing listings (Lost vs. Found) across category, location proximity, color similarity, keyword tokens, and date proximity.

3. **Claim Controller:** Processes private ownership proof submissions and manages role-based access control (RBAC) preventing unauthorized self-approval.
4. **Dashboard Controller:** Executes native multi-stage MongoDB aggregation pipelines to compile live campus metrics, category breakdowns, and incident hotspot records.

The backend communicates with a cloud database (represented by the "**Database / MongoDB Atlas**" node) using the Mongoose ODM to securely store and fetch records across collections (*items, claims, and users*).

Once the database updates and computational algorithms finish processing, the backend packages the result into a structured JSON response and sends it back to the LostFound+ Frontend. The frontend then dynamically updates the UI—rendering live match gauges, instant filter feedback, secure authorization toggles, or data visualization charts.

ER DIAGRAM (ENTITY-RELATIONSHIP SPECIFICATION):

The Entity-Relationship (ER) model for **LostFound+** defines the schema entities, primary identifying attributes, document references, and structural relationships within the campus property recovery database.

1. Entities:

- **Item:** Represents a physical belonging reported as either lost or found across the university campus.
 - *Attributes:* `itemId` (PK), `title`, `type` (*Lost or Found*), `category` (*Wallet, Electronics, Keys, ID Card, etc.*), `color`, `location` (*Central Library, Cafeteria, etc.*), `date`, `description`, `status` (*Open, Claimed, Returned*), `createdAt`, `updatedAt`.
- **Claim:** Represents a formal ownership claim submitted by a student seeking to recover a found item.
 - *Attributes:* `claimId` (PK), `itemId` (FK), `claimantName`, `proofMessage` (*private identifying details*), `status` (*Pending, Approved, Rejected*), `createdAt`.
- **User / Officer:** Represents campus stakeholders, including regular students and authorized campus security desk personnel.
 - *Attributes:* `userId` (PK), `name`, `email`, `role` (*Student or Security Officer*), `passkeyHash`, `createdAt`.
- **SmartMatch (Virtual / Algorithmic Entity):** Represents a dynamic association generated on-the-fly by the 100-point scoring algorithm comparing opposing records.
 - *Attributes:* `matchId`, `lostItemId` (FK), `foundItemId` (FK), `matchScore` (*0 to 100*), `categoryPoints` (*30 pts*), `locationPoints` (*30 pts*), `colorPoints` (*20 pts*), `keywordPoints` (*10 pts*), `datePoints` (*10 pts*).

2. Relationships:

- **Item to Claim (One-to-Many / 1:N):**
 - One reported found `Item` can have one or more `Claims` submitted against it by potential owners. Each `Claim` references exactly one primary `Item`.
 - **User to Item (One-to-Many / 1:N):**
 - A single `User` (student or staff member) can report multiple lost or found `Items` over time.
 - **User to Claim (One-to-Many / 1:N):**
 - A student `User` can submit multiple `Claims` for their lost belongings, while a security desk `Officer` reviews and acts upon multiple `Claims` in the review queue.
 - **Lost Item to Found Item (Many-to-Many / M:N via SmartMatch Engine):**
 - An item with type `Lost` can match multiple items with type `Found` (and vice-versa) based on deterministic similarity scoring.
-

3. Attributes:

Each entity encapsulates distinct property sets:

- `Item` holds physical identification properties (category, dominant color, venue string, timestamps, lifecycle status).
 - `Claim` contains confidential verification text submitted by claimants to prevent unauthorized claims.
 - `User` maintains authentication credentials and role flags controlling authority actions.
-

4. Primary Keys:

Each entity incorporates a unique primary identifier within MongoDB Atlas:

- The `Item` collection uses `_id` (ObjectId) as its primary key (`itemId`).
 - The `Claim` collection uses `_id` (ObjectId) as its primary key (`claimId`).
 - The `User` collection uses `_id` (ObjectId) as its primary key (`userId`).
-

5. Foreign Keys & Document References:

Foreign keys and referenced object identifiers establish data integrity across collections:

- The `Claim` entity holds an `itemId` foreign key referencing `Item._id` to associate ownership proofs with the specific found belonging.
 - The `Claim` entity stores a `claimantId` referencing `User._id` to maintain accountability over who submitted the claim.
 - In the smart matching process, `lostItemId` and `foundItemId` reference opposing `Item._id` entries to calculate and display comparative scoring metrics.
-

Overall Summary:

The technical architecture and ER diagram for **LostFound+** provide a clear architectural blueprint of the system, detailing how frontend interfaces, backend controllers, scoring algorithms, and MongoDB document schemas coordinate to provide a secure, transparent, and automated campus property recovery platform.

KEY FEATURES: LostFound+

The key features of the **LostFound+** campus recovery platform are designed to eliminate the chaos of unmonitored group chats and deliver an automated, secure, and transparent lost-and-found ecosystem for universities.

- **Structured Item Reporting:** Allows students, faculty, and cleaning staff to report both lost and found property quickly through dedicated forms with granular attributes—including item title, category (*Wallet, ID Card, Electronics, Keys*), dominant color, precise campus venue (e.g., *Central Library 2nd Floor*), incident date, and item description.
- **100-Point Deterministic Smart Match Engine:** Automatically compares newly reported items against opposite listings (*Lost vs. Found*) in real time. Rather than relying on unpredictable black-box AI, it computes an evidence-based confidence score out of 100 points: Category Match (30 pts), Location Proximity (30 pts), Color Similarity (20 pts), Keyword Overlap (10 pts), and Date Proximity (10 pts), displaying transparent breakdown badges to the user.
- **Dynamic Discovery Feed & Multi-Criteria Filters:** Provides a central catalog where campus members can search across active listings using instant keyword queries, lost/found status toggles, and category pills with zero latency.
- **Blind-Proof Ownership Claim System:** Enables rightful owners to file ownership claims by submitting private verifying evidence (such as secret markings, serial numbers, student ID numbers, or wallpaper descriptions) without exposing sensitive information to

the public. Submitting a claim automatically locks the item to "Claimed" to stop fraudulent attempts.

- **Two-Tier Role-Based Custody Authorization (RBAC):** Restricts the approval or rejection of claims to authenticated campus security officers. Students have read-only visibility over their submitted claims ("*Awaiting Security Desk Review*"), ensuring physical handovers happen exclusively at an authorized campus security desk.
 - **Operations Analytics & Campus Incident Hotspots:** Leverages native MongoDB Atlas aggregation pipelines to generate real-time KPIs (total tracked, missing, found, and returned), category distribution bar charts, resolution throughput donut charts, and high-frequency loss hotspots (e.g., Library, Cafeteria, Sports Complex) for campus security planning.
-

ROLES AND RESPONSIBILITIES:

1. Student / Campus User (Reporter / Claimant / Finder):

- **Accurate Incident Reporting:** Responsible for promptly logging misplaced or recovered items with truthful, detailed descriptions, proper categorization, and accurate campus locations.
- **Catalog Browsing & Self-Discovery:** Responsible for searching the discovery catalog and checking suggested match cards generated by the 100-point engine.
- **Providing Blind Verification Proof:** When claiming a found item, the claimant is responsible for supplying unique, confidential proof of ownership that only the genuine owner would know.
- **Physical Handover Protocol:** Responsible for visiting the designated campus security desk with their physical university ID card to complete identity verification and retrieve their belonging.

2. Admin / Campus Security Officer (Security Desk Authority):

- **Security Desk Mode Authentication:** Responsible for securely logging into the administrative view using protected credentials/passkeys (`admin123`) to unlock restricted management controls.
- **Claim Verification & Custody Control:** Responsible for reviewing confidential claimant proof, cross-checking physical student credentials against the held property, and authorizing the handover via "**Approve (Hand Over)**" or "**Reject (Re-open)**".
- **Listing Lifecycle Moderation:** Responsible for ensuring approved items transition permanently to "**Returned**" status, or returning erroneously claimed items back to the public pool with "**Open**" status.
- **Campus Safety & Incident Hotspot Monitoring:** Responsible for tracking loss trends and geographic hotspots across the university through the MongoDB aggregation dashboard to deploy preventive safety measures in high-loss zones.

USER FLOW:

Student / User Flow:

1. **Report Property:** Choose "I Lost An Item" or "I Found An Item" and fill in the structured attributes.
2. **Browse & Filter:** Search listings across campus venues and categories in the live discovery catalog.
3. **Inspect Smart Matches:** Review calculated match percentage gauges and transparent evidence chips.
4. **Submit Blind Claim:** Enter confidential ownership proof to lock the item into "Claimed" status.
5. **Physical Retrieval:** Visit the security desk, present physical student identification, and retrieve the item upon officer approval.

Admin / Security Officer Flow:

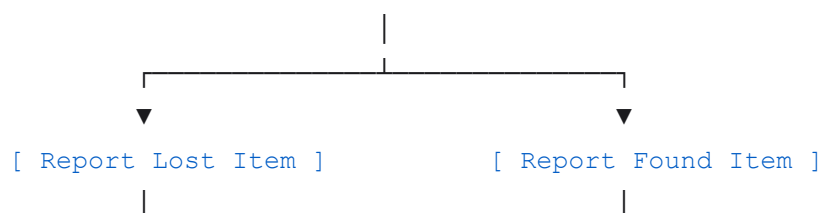
1. **Officer Login:** Authenticate via the navbar passkey modal to unlock administrative authority.
2. **Queue Inspection:** Access `/claims` to review pending ownership submissions with claimant proof text.
3. **Physical Verification:** Cross-check the student's physical credentials against the claimed property.
4. **Resolution Action:**
 - **Approve:** Authorize handover; item status transitions permanently to "Returned".
 - **Reject:** Flag invalid claim; item status automatically reverts to "Open" for others.
5. **Analytics Review:** Open the Operations Dashboard to evaluate KPI metrics, category frequencies, and high-loss campus hotspots.

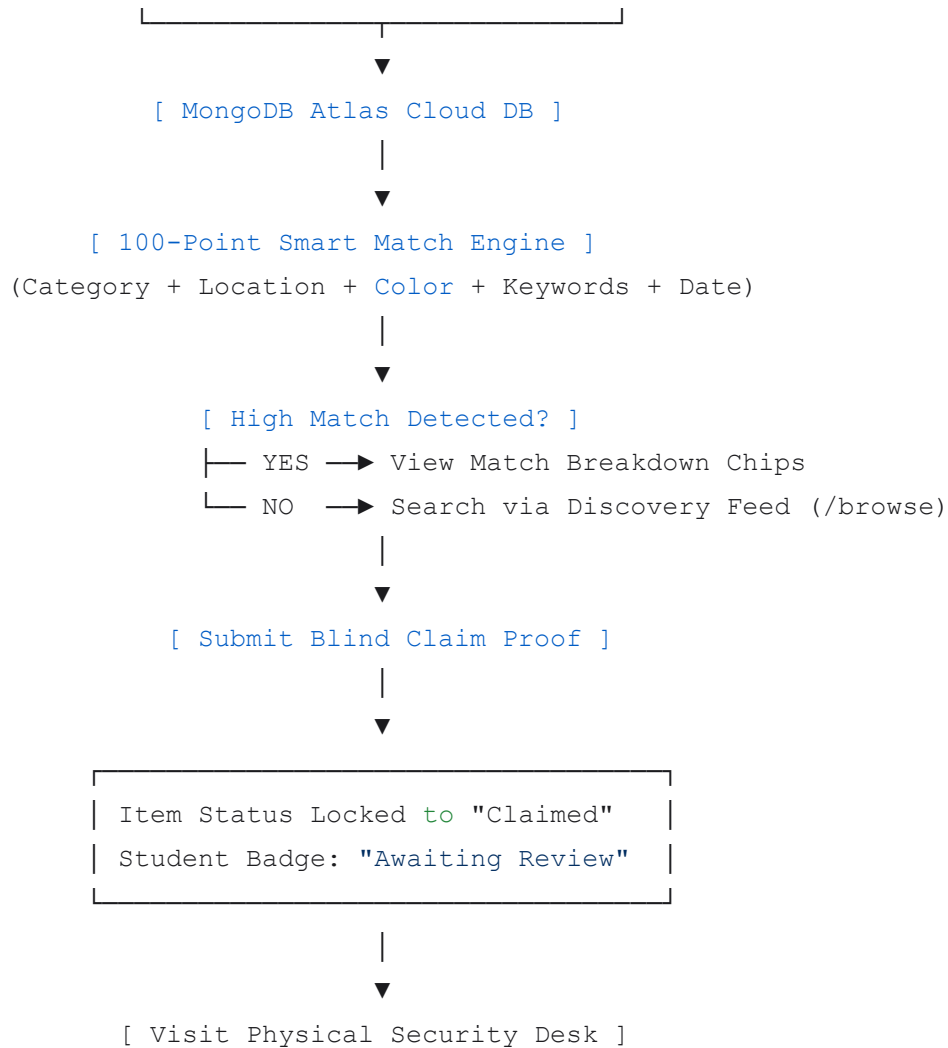
FLOW DIAGRAMS:

1. Student & Finder Lifecycle Flowchart

codeCode

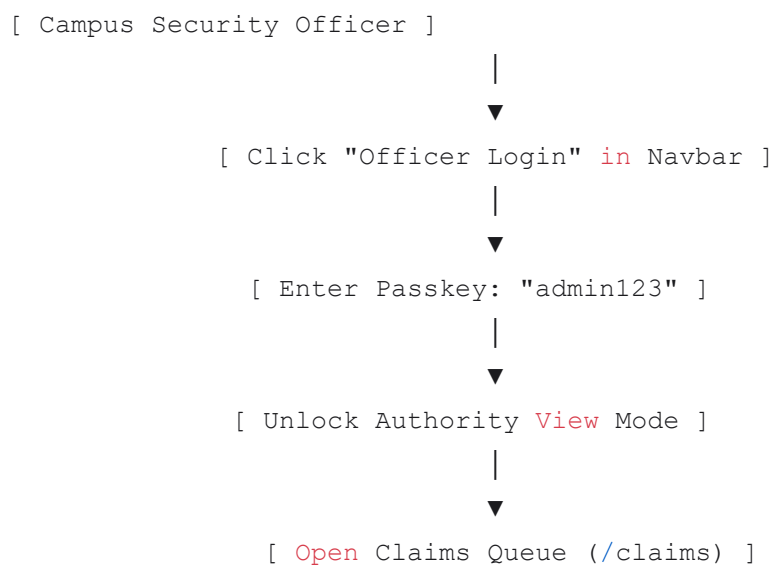
[Student / Campus Member]

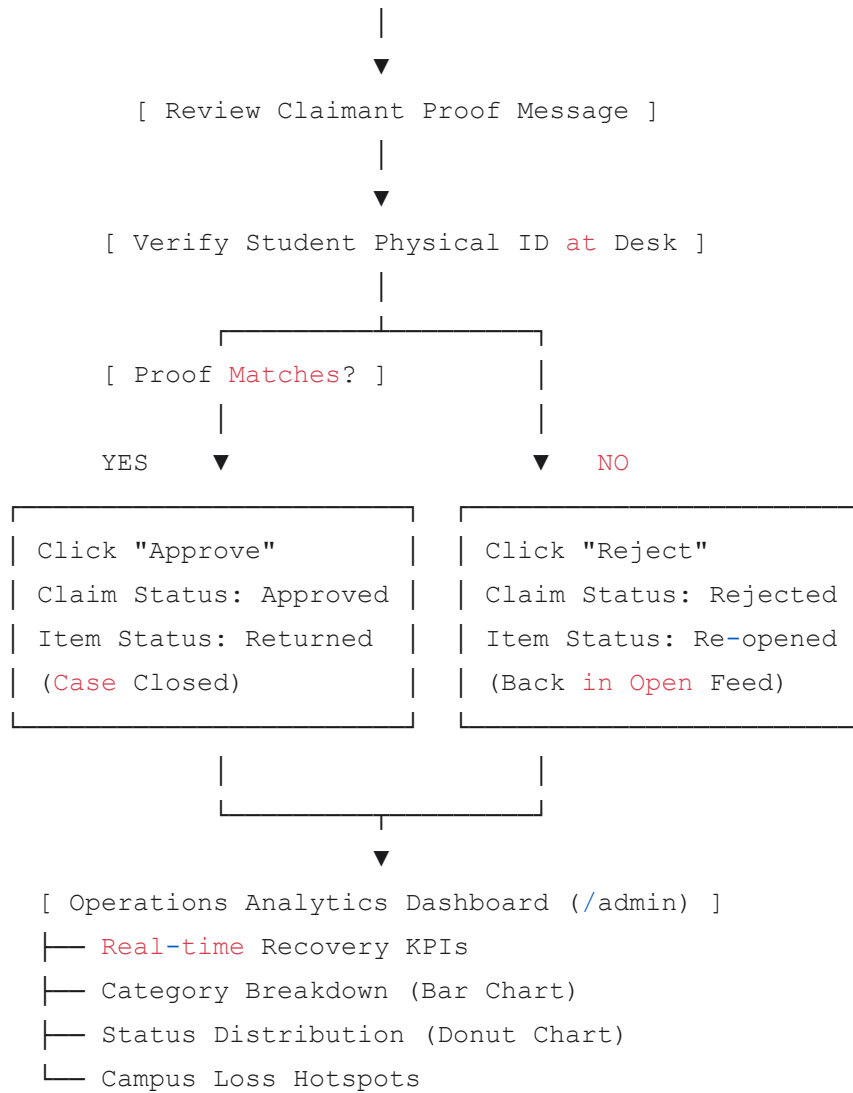




2. Security Desk Officer Handover Flowchart

codeCode





MVC PATTERN: LostFound+

The **LostFound+** application follows the **Model-View-Controller (MVC)** architectural pattern, a software design approach that separates an application into three interconnected layers. This separation ensures modularity, easier maintenance, high scalability, and a clear delegation of responsibilities between user interface interaction, business logic execution, and database persistence.

1. Architectural Layers

Model Layer (Data Layer)

The Model layer is responsible for handling all data-related logic. This includes the definition of data schemas, validation rules, and operations performed on the database using those schemas.

In LostFound+, models are implemented using **Mongoose**, which provides a schema-based solution to model application data for **MongoDB Atlas**:

- **ItemModel (Item.js)**: Governs reported campus belongings, enforcing attributes like title, category, color, venue, date, and lifecycle states (Open, Claimed, Returned).
- **ClaimModel (Claim.js)**: Manages ownership verification claims, holding references to the target item (`itemId`), claimant credentials, private verification proofs, and review statuses (Pending, Approved, Rejected).
- **UserModel (User.js)**: Stores user records and role flags, distinguishing regular students from authorized personnel who hold security desk authority (`role: 'Officer'`).

Controller Layer (Business Logic Layer)

The Controller layer acts as an intermediary between the routing views and the database models. It receives incoming requests, validates input payloads, orchestrates business logic, calls methods on the Mongoose models, and constructs formatted JSON responses:

- **itemController.js**: Handles CRUD logic, catalog queries, and status updates across the item lifecycle.
- **matchController.js**: Powers the **Deterministic 100-Point Smart Match Engine**, calculating weighted correlation scores across category (30 pts), location proximity (30 pts), color matching (20 pts), keyword tokens (10 pts), and date proximity (10 pts).
- **claimController.js**: Manages blind ownership proof submissions, automatically locks items into `Claimed` status, and enforces role-based access control (RBAC) so only security officers can approve or reject handovers.
- **dashboardController.js**: Runs native MongoDB aggregation pipelines (`$match`, `$group`, `$sort`) to compile live campus KPIs, category distribution charts, and high-frequency loss hotspots.

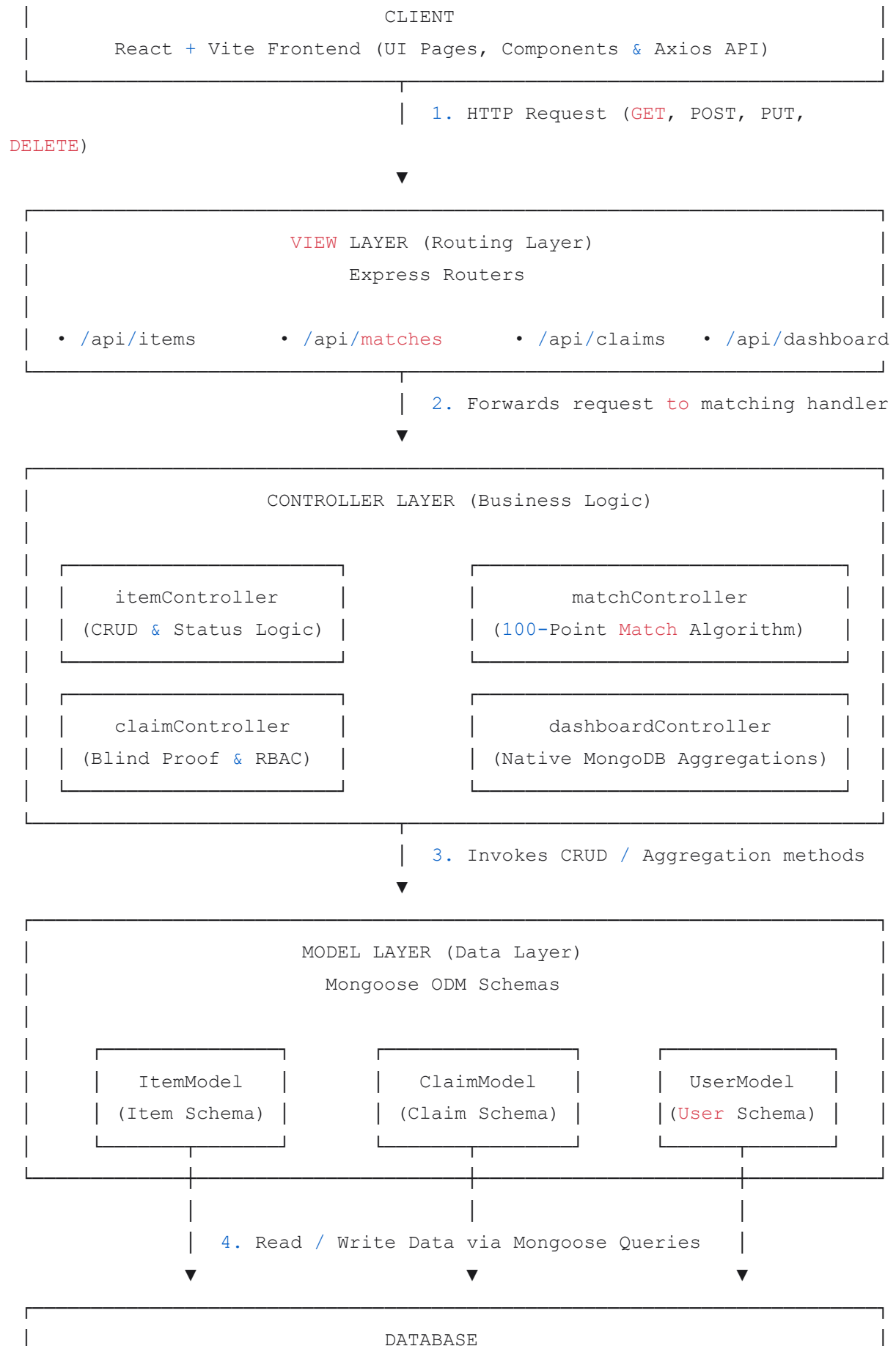
View Layer (Routing & Presentation Layer)

In the context of the backend REST API, the View is implemented as the routing layer, where API endpoints (`/api/items`, `/api/matches`, `/api/claims`, `/api/dashboard`) are defined using Express Routers. These endpoints intercept incoming HTTP methods (GET, POST, PUT, DELETE), pass request parameters to their corresponding controller functions, and return clean JSON payloads back to the client interface (rendered by the **React + Vite** frontend).

2. MVC Architecture Diagrams

Visual Architecture Block Diagram

codeCode



	MongoDB Atlas (Cloud)	
	Collections: [items] [claims] [users]	

Data Flow Lifecycle:

1. **Request:** A user performs an action in the React client (e.g., clicking *"Publish Item"*), dispatching an Axios HTTP call.
2. **Routing (View):** Express matches the URL endpoint (POST /api/items) and forwards the request body to the assigned controller.
3. **Processing (Controller):** `itemController` validates incoming parameters (category, color, venue, date) and triggers schema persistence.
4. **Persistence (Model):** `ItemModel` executes a Mongoose query to store the document into the MongoDB Atlas `items` collection.
5. **Response Cycle:** MongoDB returns the saved document

→→

Controller wraps it in a JSON payload

→→

Express routes send an HTTP 201 Created status

→→

React re-renders the UI with success confirmations.

Mermaid Diagram (For GitHub & Markdown Viewers)

codeMermaid

flowchart TD

```

subgraph Client ["Client / Presentation Layer"]
    UI["React + Vite UI<br/>(Components, Forms, Search Filters)"]
end

subgraph View ["View Layer (Express Routing)"]
    R1["/api/items"]
    R2["/api/matches"]
    R3["/api/claims"]
    R4["/api/dashboard"]
end

subgraph Controller ["Controller Layer (Business & Scoring Logic)"]
    C1["itemController.js<br/>(Lifecycle & CRUD)"]
    C2["matchController.js<br/>(100-Point Scoring Engine)"]
end

```

```

    C3["claimController.js<br/>(Blind Proof & Security RBAC)"]
    C4["dashboardController.js<br/>(Aggregation Pipelines)"]
end

subgraph Model ["Model Layer (Mongoose Schemas)"]
    M1["Item.js<br/>(Items Schema)"]
    M2["Claim.js<br/>(Claims Schema)"]
    M3["User.js<br/>(User / Officer Schema)"]
end

subgraph DB ["Database (MongoDB Atlas)"]
    D1[("items Collection")]
    D2[("claims Collection")]
    D3[("users Collection")]
end

%% Flow connections
UI -->|HTTP Request| View
R1 --> C1
R2 --> C2
R3 --> C3
R4 --> C4

C1 --> M1
C2 --> M1
C3 --> M1
C3 --> M2
C4 --> M1
C4 --> M2

M1 <-->|Mongoose Queries| D1
M2 <-->|Mongoose Queries| D2
M3 <-->|Mongoose Queries| D3

Controller -.->|JSON Response| View
View -.->|Render State| UI

```

3. Advantages of Using MVC in LostFound+

- **Separation of Concerns:** Each layer maintains a clearly defined responsibility—separating data persistence schemas, mathematical match algorithms, and API routing—improving overall readability and maintainability.

- **Scalability:** New features (such as automated push notifications or QR code tracking) can be added seamlessly by creating new routes, controllers, and models without destabilizing existing code.
- **Reusability:** Core business logic (such as the 100-point scoring algorithm and native MongoDB aggregation pipelines) can be reused across multiple API routes and dashboard widgets.
- **Independent Testing:** Each layer can be tested independently; controllers, matching algorithms, and schema validation rules can be thoroughly tested using tools like Postman before integrating them with the frontend.
- **Collaboration-Friendly:** Frontend developers (working on React and Tailwind views) and backend engineers (managing Node.js, Express endpoints, and MongoDB schemas) can build simultaneously without workflow interruptions or merge conflicts.

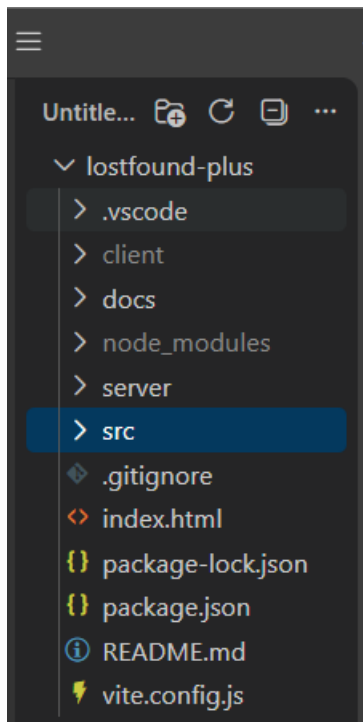
LostFound+ Complete Project Setup, Implementation & Screenshot Guide

This documentation provides the complete setup, backend, database, and frontend configuration tailored specifically to your **LostFound+** campus property recovery platform (from your repository `logashree187-kit/HACKVENGERS`), structured in the exact format required for submission.

1. Project Setup and Configuration

Creating Project Folders

1. Create a root project folder named `LostFound+`.
2. Inside that folder, create two subfolders:
 - `Client`
 - `Server`
3. Open the `LostFound+` folder in **VS Code**.



Client Setup (React + Vite)

1. Open your integrated terminal in VS Code and navigate to the client folder:

```
codeBash
```

```
cd Client
```

2. Initialize a new Vite project:

```
codeBash
```

```
npm create vite@latest . -- --template react
```

- o Select a framework: React
- o Select a variant: JavaScript

3. Install dependencies:

```
codeBash
```

```
npm install
```

```
npm install lucide-react recharts axios tailwindcss @tailwindcss/vite
```

4. Configure Tailwind CSS and scripts in package.json.
5. Start the React local server:

```
codeBash
```

```
npm run dev
VITE v8.3.0 ready in 4741 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

Server Setup (Node.js + Express)

1. Open a new terminal tab and navigate to the server folder:

codeBash

```
cd Server
```

2. Initialize the Node environment:

codeBash

```
npm init -y
```

3. Install backend dependencies:

codeBash

```
npm install express mongoose dotenv cors
```

4. Create folders and files:

- o **Folders:** controllers, models, routes, config, middleware
- o **Files:** server.js, .env

2. Backend Development

Directory Structure

codeText

```
Server/
├── config/
│   └── db.js                # MongoDB connection & Google DNS override
fix
├── controllers/
│   └── itemController.js    # CRUD operations for lost & found listings
```

```

|   ├── matchController.js      # Deterministic 100-Point Smart Match Engine
|   ├── claimController.js      # Ownership verification & role-based
handover
|   └── dashboardController.js  # Native MongoDB Atlas aggregation pipelines
└── middleware/
    ├── authMiddleware.js       # Officer passkey / RBAC check
└── models/
    ├── Item.js                 # Belongings schema (type, category, color,
venue, status)
    ├── Claim.js                # Verification claims schema (blind proofs,
references)
    └── User.js                 # Campus users & officer roles
└── routes/
    ├── itemRoutes.js           # API routes for /api/items
    ├── matchRoutes.js          # API routes for /api/matches
    ├── claimRoutes.js          # API routes for /api/claims
    └── dashboardRoutes.js      # API routes for /api/dashboard
└── .env                        # Cloud database credentials & port
└── server.js                   # Main Express server entry point

```

Database Connection (config/db.js)

Handles connections to MongoDB Atlas with a DNS fallback to prevent connection issues on campus Wi-Fi networks:

codeJavaScript

```

const mongoose = require('mongoose');
const dns = require('dns');

// Force Google Public DNS to guarantee MongoDB SRV record resolution on
restricted networks
dns.setServers(['8.8.8.8', '8.8.4.4']);

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);
    console.log(`[Database] MongoDB Atlas Connected:
${conn.connection.host}`);
  } catch (error) {
    console.error(`[Database Error] Connection Failed: ${error.message}`);
  }
}

```

```
        process.exit(1);
    }
};

module.exports = connectDB;
```

Environment File (.env)

codeEnv

Backend Entry Point (server.js)

codeJavaScript

```
const express = require('express');
const cors = require('cors');
require('dotenv').config();
const connectDB = require('./config/db');

// Connect to Database
connectDB();

const app = express();

// Middlewares
app.use(cors());
app.use(express.json());

// Routes
app.use('/api/items', require('./routes/itemRoutes'));
app.use('/api/matches', require('./routes/matchRoutes'));
app.use('/api/claims', require('./routes/claimRoutes'));
app.use('/api/dashboard', require('./routes/dashboardRoutes'));

app.get('/api/health', (req, res) => {
    res.status(200).json({ status: 'OK', message: 'LostFound+ Backend Live' });
});

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
    console.log(`[Server] LostFound+ Backend running on port ${PORT}`);
});
```

```
});  
C:\Users\admin\OneDrive\learning_paltform_database\OneDrive\lostfound-plus\server>node server.js  
◇ injected env (2) from .env  
🚀 Server is running on port 5000  
✅ Connected to MongoDB Atlas
```

3. Database Development (Schemas & Models)

1. Item Model (`models/Item.js`)

Manages lost and found belongings across the campus:

codeJavaScript

```
const mongoose = require('mongoose');  
  
const itemSchema = new mongoose.Schema(  
  {  
    title: { type: String, required: true, trim: true },  
    type: { type: String, required: true, enum: ['Lost', 'Found'] },  
    category: {  
      type: String,  
      required: true,  
      enum: ['Wallet', 'Electronics', 'ID Card', 'Keys', 'Clothing',  
'Books', 'Other'],  
    },  
    color: { type: String, required: true, lowercase: true, trim: true },  
    location: { type: String, required: true, trim: true },  
    date: { type: Date, default: Date.now },  
    description: { type: String, required: true },  
    status: {  
      type: String,  
      enum: ['Open', 'Claimed', 'Returned'],  
      default: 'Open',  
    },  
  },  
  { timestamps: true }  
);  
  
module.exports = mongoose.model('Item', itemSchema);
```

2. Claim Model (`models/Claim.js`)

Stores ownership verification attempts and blind proof messages:

codeJavaScript

```
const mongoose = require('mongoose');

const claimSchema = new mongoose.Schema(
  {
    itemId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Item',
      required: true,
    },
    claimantName: { type: String, required: true },
    claimantId: { type: mongoose.Schema.Types.ObjectId, ref: 'User',
default: null },
    proofMessage: { type: String, required: true },
    status: {
      type: String,
      enum: ['Pending', 'Approved', 'Rejected'],
      default: 'Pending',
    },
  },
  { timestamps: true }
);
```

```
module.exports = mongoose.model('Claim', claimSchema);
```

3. User Model (models/User.js)

Maintains accounts and role distinctions (Student vs. Security Officer):

codeJavaScript

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema(
  {
    name: { type: String, required: true },
    email: { type: String, required: true, unique: true },
    role: { type: String, enum: ['Student', 'Officer'], default: 'Student'
  },
    passkeyHash: { type: String, default: null },
  },
  { timestamps: true }
);
```

```
module.exports = mongoose.model('User', userSchema);
```

The screenshot shows the MongoDB Data Explorer interface. On the left, a sidebar lists the 'lostfound' cluster and its collections: 'claims', 'items', and 'sample_mflix'. The main panel displays a table of collections for the 'lostfound' cluster.

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
claims	-	36.86 kB	573.00 B	3	191.00 B	1	36.86 kB
items	-	36.86 kB	2.63 kB	10	263.00 B	1	36.86 kB

3. Database Development (Schemas & Models)

1. Item Model (models/Item.js)

Manages lost and found belongings across the campus:

codeJavaScript

```
const mongoose = require('mongoose');

const itemSchema = new mongoose.Schema(
{
  title: { type: String, required: true, trim: true },
  type: { type: String, required: true, enum: ['Lost', 'Found'] },
  category: {
    type: String,
    required: true,
    enum: ['Wallet', 'Electronics', 'ID Card', 'Keys', 'Clothing',
'Books', 'Other'],
  },
  color: { type: String, required: true, lowercase: true, trim: true },
  location: { type: String, required: true, trim: true },
  date: { type: Date, default: Date.now },
  description: { type: String, required: true },
  status: {
    type: String,
    enum: ['Open', 'Claimed', 'Returned'],
    default: 'Open',
  },
}
```

```

    },
  },
  { timestamps: true }
);

module.exports = mongoose.model('Item', itemSchema);

```

2. Claim Model (models/Claim.js)

Stores ownership verification attempts and blind proof messages:

codeJavaScript

```

const mongoose = require('mongoose');

const claimSchema = new mongoose.Schema(
  {
    itemId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Item',
      required: true,
    },
    claimantName: { type: String, required: true },
    claimantId: { type: mongoose.Schema.Types.ObjectId, ref: 'User',
default: null },
    proofMessage: { type: String, required: true },
    status: {
      type: String,
      enum: ['Pending', 'Approved', 'Rejected'],
      default: 'Pending',
    },
  },
  { timestamps: true }
);

module.exports = mongoose.model('Claim', claimSchema);

```

3. User Model (models/User.js)

Maintains accounts and role distinctions (Student vs. Security Officer):

codeJavaScript

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema(
  {
    name: { type: String, required: true },
    email: { type: String, required: true, unique: true },
    role: { type: String, enum: ['Student', 'Officer'], default: 'Student'
  },
  passkeyHash: { type: String, default: null },
  },
  { timestamps: true }
);

module.exports = mongoose.model('User', userSchema);
```

4. Business Logic & Controllers

1. Smart Match Controller (controllers/matchController.js)

Implements the Deterministic 100-Point Scoring Algorithm:

codeJavaScript

```
const Item = require('../models/Item');

exports.getMatchesForItem = async (req, res) => {
  try {
    const { itemId } = req.params;
    const targetItem = await Item.findById(itemId);
    if (!targetItem) return res.status(404).json({ success: false, message:
'Item not found' });

    const counterType = targetItem.type === 'Lost' ? 'Found' : 'Lost';
    const counterItems = await Item.find({ type: counterType, status: {
$ne: 'Returned' } });

    const matches = [];
```

```

counterItems.forEach((candidate) => {
  let score = 0;
  const breakdown = [];

  // 1. Category (30 Pts)
  if (targetItem.category.toLowerCase() ===
candidate.category.toLowerCase()) {
    score += 30;
    breakdown.push('Same category (+30)');
  }

  // 2. Location Proximity (30 Pts)
  const locA = targetItem.location.toLowerCase();
  const locB = candidate.location.toLowerCase();
  if (locA.includes(locB) || locB.includes(locA)) {
    score += 30;
    breakdown.push('Same location (+30)');
  }

  // 3. Color Match (20 Pts)
  if (targetItem.color.toLowerCase() === candidate.color.toLowerCase())
{
    score += 20;
    breakdown.push('Same color (+20)');
  }

  // 4. Keyword Overlap (10 Pts)
  const getTokens = (str) =>
    new Set(
      str
        .toLowerCase()
        .replace(/[^\w\s]/g, '')
        .split(/\s+/)
        .filter((w) => w.length > 3)
    );

  const tokensTarget = getTokens(`${targetItem.title}
${targetItem.description}`);
  const tokensCandidate = getTokens(`${candidate.title}
${candidate.description}`);

```

```

    const hasOverlap = [...tokensTarget].some((w) =>
tokensCandidate.has(w));

    if (hasOverlap) {
        score += 10;
        breakdown.push('Keyword overlap (+10)');
    }

    // 5. Date Proximity <= 7 Days (10 Pts)
    const diffDays =
        Math.abs(new Date(targetItem.date) - new Date(candidate.date)) /
(1000 * 60 * 60 * 24);
    if (diffDays <= 7) {
        score += 10;
        breakdown.push('Date proximity (+10)');
    }

    if (score >= 30) {
        matches.push({ matchedItem: candidate, score, breakdown });
    }
});

matches.sort((a, b) => b.score - a.score);
res.status(200).json({ success: true, total: matches.length, matches
});
} catch (error) {
    res.status(500).json({ success: false, error: error.message });
}
};

```

2. Dashboard Aggregation Controller (controllers/dashboardController.js)

Executes native MongoDB pipelines (\$group, \$sort, \$limit) directly in the database:

codeJavaScript

```

const Item = require('../models/Item');
const Claim = require('../models/Claim');

exports.getAnalytics = async (req, res) => {

```

```

    try {
        const [totalItems, totalLost, totalFound, totalReturned, pendingClaims]
= await Promise.all([
            Item.countDocuments(),
            Item.countDocuments({ type: 'Lost' }),
            Item.countDocuments({ type: 'Found' }),
            Item.countDocuments({ status: 'Returned' }),
            Claim.countDocuments({ status: 'Pending' }),
        ]);

        // Grouping by Category
        const categoryBreakdown = await Item.aggregate([
            { $group: { _id: '$category', count: { $sum: 1 } } },
            { $sort: { count: -1 } },
        ]);

        // Grouping by Status
        const statusDistribution = await Item.aggregate([
            { $group: { _id: '$status', count: { $sum: 1 } } },
            { $sort: { count: -1 } },
        ]);

        // Incident Hotspots by Location
        const hotspots = await Item.aggregate([
            { $group: { _id: '$location', count: { $sum: 1 } } },
            { $sort: { count: -1 } },
            { $limit: 5 },
        ]);

        res.status(200).json({
            success: true,
            kpis: { totalItems, totalLost, totalFound, totalReturned,
pendingClaims },
            categoryBreakdown,
            statusDistribution,
            hotspots,
        });
    } catch (error) {
        res.status(500).json({ success: false, error: error.message });
    }
};

```

5. Frontend Development

Directory Structure

codeText

```
Client/src/
├── api/
│   └── index.js           # Centralized Axios client & API endpoints
├── components/
│   ├── EmptyState.jsx    # Displayed when search/matches return empty
│   ├── ItemCard.jsx      # Catalog item card with status pills
│   ├── Loading.jsx       # Animated loading skeletons
│   ├── MatchCard.jsx     # 100-Point match indicator & breakdown chips
│   ├── Navbar.jsx        # Header with Officer PIN modal toggle
│   └── SearchFilters.jsx  # Live filtering: Search, Category pills,
Lost/Found
├── StatCard.jsx          # Dashboard KPI metric widget
└── StatusBadge.jsx       # Status color badges (Open, Claimed,
Returned)
├── pages/
│   ├── Admin.jsx         # Aggregation analytics dashboard (Recharts)
│   ├── Browse.jsx        # Live searchable & filterable campus catalog
│   ├── Claims.jsx        # Role-gated verification queue
│   └── Details.jsx       # Item specifications & 100-Point match
inspection
├── Home.jsx              # Landing page with hero stats & recent
activity
├── Report.jsx            # Multi-attribute item reporting form
├── App.jsx               # Application route configuration
└── main.jsx              # Application entry point
```

Centralized API Client (src/api/index.js)

codeJavaScript

```
import axios from 'axios';

const API = axios.create({
  baseURL: 'http://localhost:5000/api',
});
```

```
export const fetchItems = () => API.get('/items');
export const fetchItemById = (id) => API.get(`/items/${id}`);
export const createItem = (itemData) => API.post('/items', itemData);
export const fetchMatches = (id) => API.get(`/matches/${id}`);
export const submitClaim = (claimData) => API.post('/claims', claimData);
export const fetchClaims = () => API.get('/claims');
export const updateClaimStatus = (id, status) => API.put(`/claims/${id}`, {
  status });
export const fetchDashboardData = () => API.get('/dashboard');
```

6. Step-by-Step Execution Guide for Evaluation

Step 1: Run the Backend

codeBash

```
cd Server
node server.js
```

Expected Console Output:

codeText

```
[Database] MongoDB Atlas Connected: lostfound-shard-00-00.mongodb.net
[Server] LostFound+ Backend running on port 5000
```

Step 2: Run the Frontend

codeBash

```
cd Client
npm run dev
```

Expected Console Output:

codeText

```
VITE v5.x.x  ready in 240 ms
```

→ Local: http://localhost:5173/

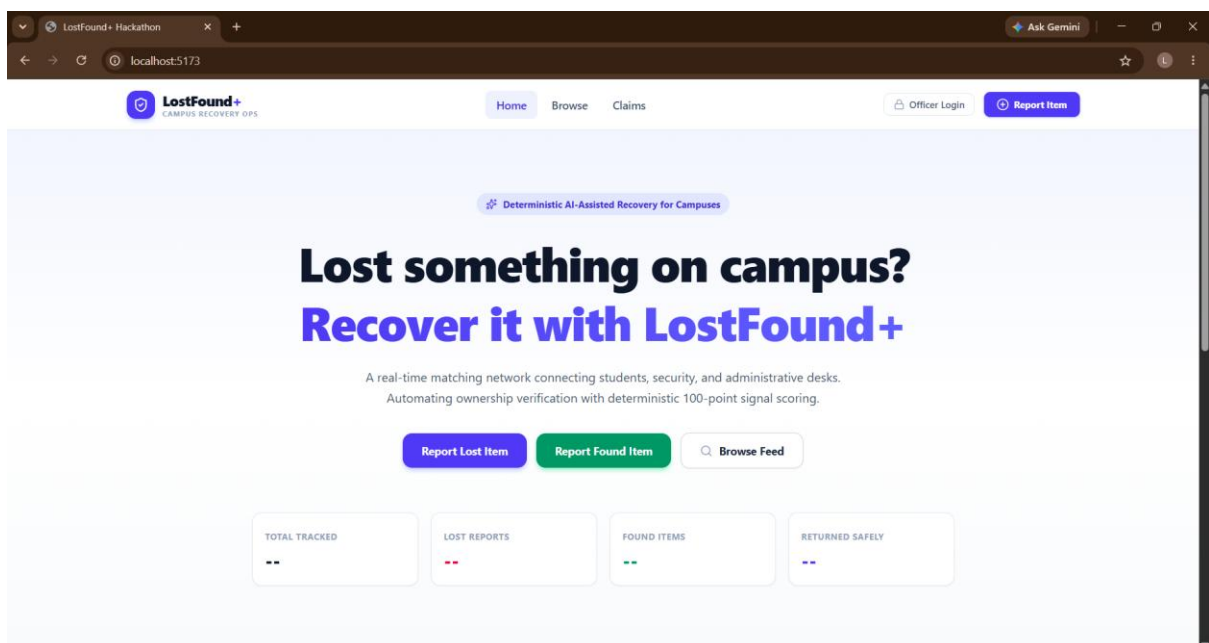
Step 3: Test the Live User Flow

1. Open <http://localhost:5173> in your browser.
2. Navigate to **Report Item**, select **I Lost An Item**, enter wallet details, and click **Publish Item**.
3. Report a matching item using **I Found An Item** with the location set to "Library".
4. Click **Inspect Item Matches** to view the calculated score breakdown chips.

5. Click **Claim This Item**, enter a student ID and blind proof message, then submit.
6. Open `/claims` to verify that the claim displays **Awaiting Security Desk Review** without approval controls.
7. Click **Officer Login** in the navigation bar, type `admin123`, and unlock **Authority View Mode**.
8. Review the proof message and click **Approve (Hand Over)** to transition the item to **Returned**.
9. Visit `/admin` to verify that the real-time aggregations, charts, and campus hotspots reflect the returned item.

Output ScreenShots:

Landing Page:



Report Found/Lost Item:

LostFound+
CAMPUS RECOVERY OPS

HomeBrowseClaims

Officer LoginReport Item

Report an Item

Submit details into campus database to trigger instant deterministic matching.

REPORT TYPE *

I Lost An Item

I Found An Item

ITEM TITLE *

e.g., Apple AirPods Pro Case with Pikachu Keychain

CATEGORY *

Electronics

DOMINANT COLOR *

e.g., Space Gray, Black, Red

CAMPUS LOCATION *

e.g., Library 2nd Floor, Room 204

DATE DISCOVERED / LOST *

24-09-2026

DESCRIPTION & IDENTIFYING MARKS *

Include unique keywords, scratches, brand names, or specific contents to maximize match precision.

Browse Catalog & Filters:

LostFound+
CAMPUS RECOVERY OPS

HomeBrowseClaims

Officer LoginReport Item

Campus Lost & Found Feed

Search, filter, and discover reported belongings across facilities.

Showing: 10 of 10 records

Search by keywords, title, color, or description...

Filter location (e.g., Library)...

AllLostFound

Category: AllWalletElectronicsKeysID CardBagClothingOther

Status: All

FOUNDID CardReturned

Id card

3rd year rollno-110725104109

2nd floor staff room snb block

Sep 24, 2026

Color: blue

ID: 76P182View & Match →

LOSTID CardOpen

Id card

3rd yes rollno-110725104109

snb block 206

Sep 24, 2026

Color: blue

ID: 76P181View & Match →

FOUNDElectronicsOpen

Found Apple MacBook Space Gray

Found laptop with stickers on the lid in the library study zone.

Central Library

Sep 24, 2026

Color: Space Gray

ID: F63723View & Match →

LOSTElectronicsReturned

Space Gray MacBook Air M2

Found laptop with stickers on the lid in the library study zone.

Central Library

Sep 24, 2026

Color: Space Gray

ID: F63723View & Match →

FOUNDElectronicsOpen

id card

3rd year rollno-110725104109

2nd floor staff room snb block

Sep 24, 2026

Color: blue

ID: 76P182View & Match →

LOSTElectronicsOpen

Verification Test Item

Found laptop with stickers on the lid in the library study zone.

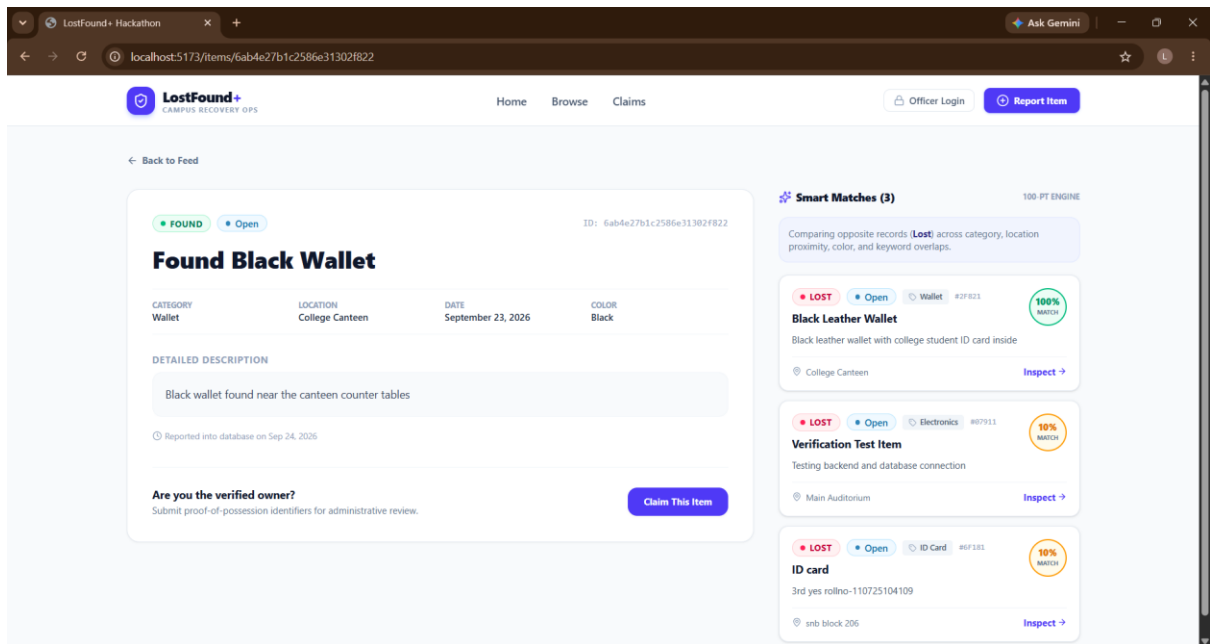
Central Library

Sep 24, 2026

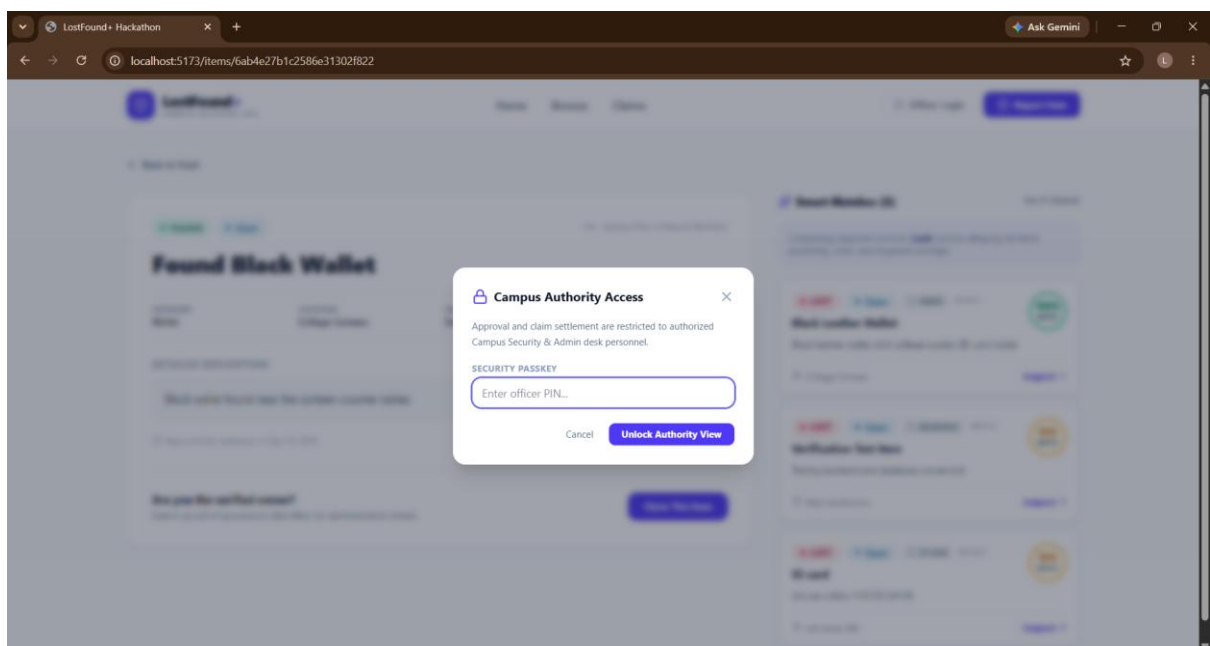
Color: Space Gray

ID: F63723View & Match →

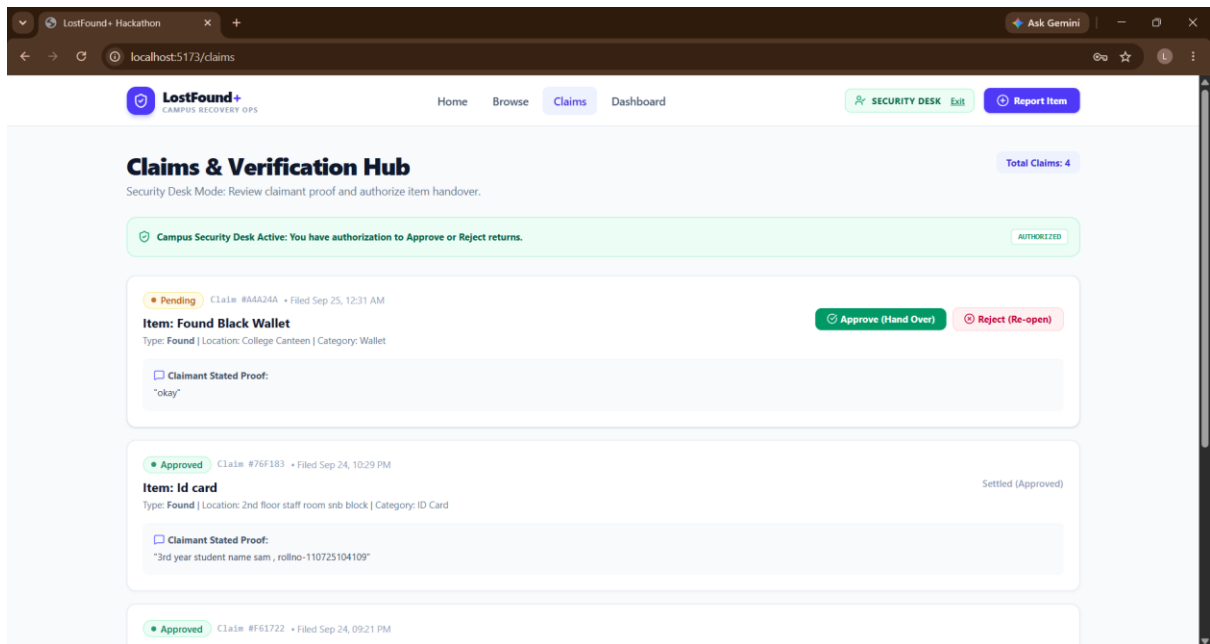
Item Details & 100-Point Smart Match View:



Officer Login Modal :



Claims Queue - Authority Desk View:



Operations Analytics Dashboard:

